

Paso 7 - Repetir con `for` | Ejercicios

`for` para cuando **sí sabes** cuántas vueltas son. Acuérdate del acumulador: una variable que nace **antes** del `for` y crece adentro.

► **Plantillas:** [plantillas/](#) — un `.java` por ejercicio, con el enunciado, la salida esperada y los `// TODO` donde va tu código. Si las usas, el archivo ya viene creado (`B7_1_Multiplos.java`) y **ese** nombre manda sobre el que diga el enunciado.

✓ **Solucionario:** [soluciones/](#) — el código de cada ejercicio, la salida real y por qué se resuelve así. Ábrelo **después** de intentarlo.

Cómo se lee cada ejercicio: el título dice qué hay que lograr; el bloque de abajo es la **salida esperada**, o sea lo que tiene que aparecer en tu pantalla (en los programas que preguntan cosas, lo que va después de la pregunta es lo que **escribe el usuario**); las citas como esta son pistas. Si no sabes por dónde partir, abre la plantilla: los `// TODO` numerados son los pasos, en orden. Compila y ejecuta después de cada uno.

B7.1 - Los múltiplos de 3

Qué practicas: recorrer un rango con `for`, acumular una cuenta y una suma, y controlar cuántos valores van por línea.

Sin pedir nada por teclado, recorre del 1 al 100 y muestra cuántos múltiplos de 3 hay y cuánto suman.

```
Multiplos de 3 entre 1 y 100: 33
Suman: 1683
```

Después imprímelos, 10 por línea:

```
3 6 9 12 15 18 21 24 27 30
33 36 39 42 45 48 51 54 57 60
...
```

Pista para las 10 por línea: un `if` con un contador y `System.out.println()`.

B7.2 - El factorial

Qué practicas: un acumulador que multiplica en vez de sumar, y el desbordamiento silencioso de `int` frente a `long`.

Pide un número del 1 al 15 y muestra su factorial (`5! = 1 * 2 * 3 * 4 * 5 = 120`).

```
Numero: 5
5! = 120
```

Ahora prueba con 13, guardando el resultado en un `int`:

```
13! = 1932053504
```

Está **mal**: 13! es 6227020800. No cabe en un `int` y Java no avisa nada, solo entrega un número inventado. Con 20 sale hasta negativo (`-2102132736`). Cámbialo a `long` y vuelve a probar: aguanta hasta el 20 y se rompe en el 21.

```
Un int llega hasta 2.147.483.647. Puedes verlo con
System.out.println(Integer.MAX_VALUE);
```

B7.3 - Las figuras

Qué prácticas: `for` anidados, donde el ciclo externo controla las filas y el interno lo que se dibuja en cada una.

Dibuja las tres, cada una con un `for` dentro de otro. El tamaño lo pide por teclado (en los ejemplos, 4).

1. El creciente (ya lo hiciste en el paso 7):

```
*
**
***
****
```

2. El decreciente:

```
****
***
**
*
```

3. La pirámide, que necesita **dos** `for` adentro: uno que dibuja los espacios y otro los asteriscos.

```
*  
***  
*****  
*****
```

B7.4 - La tabla completa

Qué prácticas: un `for` dentro de otro para generar una cuadrícula, y `printf` para que las columnas queden alineadas.

Imprime las tablas del 1 al 10, todas, en forma de cuadrícula:

```
1  2  3  4  5  6  7  8  9 10  
2  4  6  8 10 12 14 16 18 20  
3  6  9 12 15 18 21 24 27 30  
...  
10 20 30 40 50 60 70 80 90 100
```

Pista: `System.out.printf("%3d", fila * columna);` y el `println()` al terminar cada fila.

B7.5 - Desafío: los primos

Qué prácticas: un `for` anidado con una variable `boolean` de bandera y un `break` para cortar la búsqueda apenas se sabe la respuesta.

Pide un número `n` y muestra todos los primos entre 2 y `n`.

```
Hasta que numero? 30  
2 3 5 7 11 13 17 19 23 29  
Son 10 primos
```

Un número es primo si **ningún** número entre 2 y `n - 1` lo divide exacto. Vas a necesitar un `for` dentro de otro, una variable `boolean esPrimo` y un `break` para cortar apenas encuentres un divisor.

Cuando termines, sigue con el [paso 8](#).